STM32 三节点 CAN 项目 · 深度自测题库

> 用途：把你**已经做出来但没吃透**的项目，学到能在面试里脱口而出。 用法：先**遮住答案自己答**，答不上来再看；重点盯每模块最后的 **“项目必答”**（面试官最可能问）。 覆盖面：C 语言 · FreeRTOS · PID · CAN · Bootloader/Flash。全部**结合你的真实项目**。

使用说明：什么算“真懂了”

对任何一道题，能用这 4 句话讲出来，才算过关：

1. **它解决什么问题**（为什么要有它）；
2. **它怎么实现**（关键代码/结构/流程）；
3. **为什么这么设计**（有哪些替代方案，为什么选这个）；
4. **我踩过什么坑 / 怎么验证它对。**

模块一：C 语言（嵌入式方向）

A. 基础必会

Q1：`volatile` 是干什么的？你项目哪里用了？

- 答：告诉编译器这个变量**可能被编译器看不到的东西改变**（中断、外设寄存器、其他任务），**禁止优化**掉对它的读写。
- 项目：ISR 与任务共享的标志/缓冲区、外设寄存器、DMA 传输标志。

Q2：`static` 的两个含义？

- 答：① 修饰**函数内变量**：生命周期是全程，但作用域仅函数内，初值只初始化一次；② 修饰**文件级变量/函数**：链接属性为“内部”，本文件私有，避免跨文件重名。
- 项目：`static TIM_HandleTypeDef htim2;`、`static int32_t pid_err1;`、`static QueueHandle_t xCanRxQueue;`。

Q3：`const` 的作用？修饰指针的三种写法分别是什么意思？

- 答：只读，保护不该改的数据，也帮编译器优化。
- `const char *p`：p 指向的内容不能改；
- `char * const p`：p 本身不能改（指向固定）；
- `const char * const p`：都不能改。

Q4：指针和数组有什么区别？

- 答：数组有确定的存储、`sizeof` 是整个数组大小；数组名在表达式里会**退化为指向首元素的指针**；数组不可赋值，指针可改指向；`&arr` 与 `arr` 类型不同（前者是“数组指针”）。

Q5：函数指针怎么用？你项目哪里用了？

- 答：用于**回调**和**跳转**。你项目里 Bootloader 跳 APP 就用了：`void (*reset)(void) = (void(*)(void))(*(volatile uint32_t*)(APP_START + 4)); reset();`

Q6：结构体为什么要对齐？`#pragma pack` 有什么用？

- 答：编译器默认按成员对齐，会插入填充字节（换访问效率）。`#pragma pack(1)` 取消填充，用于**协议/存储结构**。但**解析通信帧不要直接强转结构体**——会有对齐和端序问题。

Q7：宏和函数有什么区别？宏有哪些坑？

- 答：宏是**预处理展开**，无类型检查、无栈开销、代码膨胀；函数有类型、有调用开销。
- 坑：**副作用 + 缺括号**。`#define SQR(x) ((x)*(x))` 必须套括号；多语句用 `do{...}while(0)`。

Q8：怎么置位、清位、翻转、读取某一位？

- 答：置位 `x |= (1u<<n)`；清位 `x &= ~(1u<<n)`；翻转 `x ^= (1u<<n)`；读取 `(x>>n)&1u`。
- 项目：`CAN1->MCR &= ~CAN_MCR_DBF;`（清调试冻结位）、过滤器 `ID << 5`。

Q9：什么是大小端？你的协议怎么处理？

- 答：多字节数据在内存里的字节顺序。STM32 是**小端**（低字节在低地址）。
- 你的协议统一小端：组帧 `data[0]=v&0xFF; data[1]=(v>>8)&0xFF;`，解析 `v=data[0]|(data[1]<<8);`。

B. 进阶（容易被追问）

Q10：有符号/无符号混用、整数提升有什么坑？

- 答：涉及窄于 `int` 的运算会先提升为 `int`；有符号与无符号比较大小时，有符号会被转成无符号，导致“负数 > 正数”这类诡异结果。多字节比较/移位建议显式转 `int32_t/uint32_t`。

Q11：移位有什么陷阱？

- 答：移位位数 ≥ **类型位宽是未定义行为**；对有符号负数右移是实现相关（通常算术右移）；左移溢出是 UB。安全做法：用无符号、先判断位数。

Q12：什么是可重入函数？

- 答：可被多个任务/中断**同时或重入调用**都安全的函数：不依赖可写的全局/静态状态（或加锁保护），不用非可重入的库函数。

Q13：程序的内存分区有哪些？嵌入式为什么少用堆？

- 答：代码段(.text)、常量段、全局/静态区(.data 已初始化 / .bss 未初始化)、栈、堆。嵌入式少用堆是因为**碎片、分配时间不确定、失败难恢复**，实时系统要确定性，所以多用静态分配。

Q14：字符串处理要注意什么？

- 答：`strcpy/strcat` 不检查长度会**缓冲区溢出**，用 `strncpy/snprintf` 并保证结尾 `\0`；`sprintf` 同理。

C. 项目必答

Q15：你项目怎么解析 CAN 帧？为什么不用结构体直接强转？

- 答：从 `HAL_CAN_GetRxMessage` 拿到 `data[]`，**按字节偏移手动拼装**（小端）：`speed = data[0] | (data[1]<<8)`。
- 不强转的原因：结构体有**对齐填充**、且**端序**依赖平台，强转容易读错；按字节拼最稳、可移植。

Q16：帧发送时怎么组小端数据？

- 答：`data[0]=(uint8_t)(v & 0xFF); data[1]=(uint8_t)((v>>8)&0xFF);`。多帧字段（如 4 字节总长）用 `to_bytes/to_le32` 同理。

Q17：你项目用了动态内存（malloc）吗？为什么？

- 答：**没用**。FreeRTOS 用 `heap_4` 做静态/受控分配，业务里用静态缓冲和固定长度数组，避免碎片和不确定性。

Q18：中断和任务共享的变量怎么保护？

- 答：① 能用**队列/任务通知**传递就别共享；② 简单标志用 `volatile` + 临界区；③ 复杂共享用互斥量（任务间）；④ ISR 里只用 `FromISR` 接口。你项目用 `xQueueSendFromISR` 把帧投给任务，从根上避免共享。

模块二：FreeRTOS

A. 基础必会

Q1：FreeRTOS 的调度方式？任务有哪些状态？

- 答：**抢占式**（更高优先级就绪立即抢占）+ **同优先级时间片轮转**。状态：运行 / 就绪 / 阻塞 / 挂起。

Q2：优先级数字越大越高还是越低？

- 答：**数字越大优先级越高**（和中断优先级相反，注意别搞混）。

Q3：队列、信号量、互斥量、事件组、任务通知怎么选？

- 答：
  - 队列：**传数据**（你用它传 CAN 帧 `xCanRxQueue`）；
  - 二值信号量：**ISR 通知任务**“有事件了”；
  - 计数信号量：资源/事件计数；
  - 互斥量：**保护共享资源**（你用它保护 UART 打印 `uartMutex`）；

- 事件组：多条件 AND/OR 等待；
- 任务通知：最轻量、1 对 1、省内存。

Q4：ISR 里要注意什么？为什么用 `FromISR` 结尾的接口？

- 答：中断里**不能阻塞、不能 printf、不能锁普通互斥量**；只能用带 `FromISR` 的安全接口，并配合 `portYIELD_FROM_ISR` 决定是否切换任务。你项目里 CAN 中断只做“取帧 + 入队”。

Q5：二值信号量和互斥量有什么区别？

- 答：互斥量有**所有权**和**优先级继承**，用于保护资源；二值信号量用于同步（事件通知），没有所有权。

Q6：什么是优先级反转？怎么解决？

- 答：低优先级任务持锁、高优先级任务等待，此时中优先级任务抢占，导致高优先级被“饿”。
- 解决：**互斥量的优先级继承**——持锁任务被临时提升到等待者的最高优先级。

Q7：临界区是什么？`taskENTER_CRITICAL` 和 `portENTER_CRITICAL` 区别？

- 答：临界区是**关中断/屏蔽调度**保护的、不可被打断的代码段。`taskENTER_CRITICAL` 可嵌套、会记录状态；`portENTER_CRITICAL` 是更底层的移植层宏，ISR 里用 `FromISR` 版本。临界区要**尽量短**。

Q8：heap_1~heap_5 的区别？你的项目用哪个？

- 答：heap_1 只分配不释放；heap_2 按大小释放但不合并；heap_3 包装标准 malloc/free；heap_4 **能合并相邻空闲块**（推荐，你用）；heap_5 支持多块不连续内存。

Q9：栈溢出怎么检测？

- 答：开 `vApplicationStackOverflowHook` 钩子；运行时可用 `uxTaskGetStackHighWaterMark` 看**剩余栈余量**。

Q10：HardFault 怎么定位？

- 答：解析现场——用 `EXC_RETURN` 判断出错时用的是 **MSP 还是 PSP**，读压栈的 8 个寄存器（R0–R3、R12、LR、PC、xPSR），拿到 **PC** 再用 `addr2line` 定位到函数/行号；配合 `configASSERT` 收紧。

B. 进阶

Q11：任务通知和信号量比，优缺点？

- 答：任务通知**更轻量、更快、省 RAM**，但**只能 1 对 1**、只能有一个事件值；信号量/队列更通用、能 1 对多、能传数据。

Q12：软件定时器是怎么实现的？

- 答：由一个**定时器服务任务 + 命令队列**驱动；回调在服务任务上下文执行，所以回调里不能阻塞。

Q13：阻塞和挂起有什么区别？

- 答：**阻塞**有超时、会被事件唤醒（等待队列/延时）；**挂起**（vTaskSuspend）是永久移出调度，直到显式恢复，不受超时影响。

C. 项目必答

Q14：你的 CAN 接收为什么用队列，而不是中断里直接处理?

- 答：中断会**抢占所有任务**，处理越久系统抖动越大；队列让中断只花几微秒投递，解析/业务交给 `CanRxTask`，任务可阻塞不空转。这就是“**中断生产、任务消费**”。

Q15：你的任务怎么划分的？各干什么?

- 答：主控有 CAN 收发、控制逻辑、故障监控、UART 调试等任务；收发走队列、离线检测走事件/时间戳、打印用互斥量保护。核心思想是**按职责和实时性分区、低耦合。**

Q16：心跳/离线检测怎么设计的？超时几秒，为什么?

- 答：主控 1s 发一帧 0x100 心跳；从节点/主控记录**最近一次收到的 tick**，超过 **5s** 没收到判离线，恢复后自动清标志。
- 为什么 5s：≈ 5 个心跳周期，**容忍偶发丢帧**又不至于误报；状态**只在翻转时打印**，避免刷屏。

Q17：UART 打印为什么要加互斥量？ISR 里 printf 会怎样?

- 答：多个任务同时 printf 会把输出**交错、丢字**，用 `uartMutex` 串行化；ISR 里 printf 会**和任务争 UART、且耗时过长**，所以 ISR 只入队，错误详情由任务周期读 ESR 打印。

模块三：PID

A. 基础必会

Q1：P、I、D 各管什么?

- 答：P 按**当前误差**成比例输出，主导响应；I **累积历史误差**消除稳态误差；D 按**误差变化率**抑制超调，但对噪声敏感。

Q2：位置式 vs 增量式 PID？公式是什么?

- 答：
  - 位置式输出绝对量 `u = Kp·e + Ki·Σe + Kd·(e−e1)`，需累加全部历史误差，易饱和；
  - **增量式输出变化量**：`Δu = Kp(e−e1) + Ki·e + Kd(e−2e1+e2)`，`u += Δu`。无累积误差，便于限幅和切换（你项目用增量式）。

Q3：什么是积分饱和（windup）？怎么处理?

- 答：执行器到限幅后误差仍在积分，恢复时输出长时间滞留导致**大幅超调。**处理：积分限幅、**积分分离**、遇限消积分、反算抗饱和。

Q4：微分项有什么问题？怎么改善?

- 答：D 对**噪声极敏感**（会把噪声放大成抖动输出）。改善：低通滤波、**不完全微分**、微分先行（只对反馈微分）、适当减小 Kd，甚至 Kd=0。

Q5：采样/控制周期怎么选?

- 答：一般取被控对象响应时间的 1/10 量级；周期要**稳定**（你项目固定 20ms）；测速采样周期应 ≤ PID 周期，否则反馈滞后。

Q6：调参顺序?

- 答：**先 P** 找到临界震荡（等幅）→ Kp 减半留裕量；**再加 I** 消稳态误差；**最后加 D** 抑超调。**一次只动一个参数、一个方向。**

Q7：阶跃响应看哪几个指标?

- 答：上升时间、超调量、稳态误差、是否等幅震荡。等幅震荡说明**增益过大。**

B. 进阶

Q8：积分分离、积分限幅、遇限消积分、反算抗饱和的区别?

- 答：积分分离——`|e|` 大时**暂时关掉 I**；积分限幅——限制积分项上下界；遇限消积分——输出饱和时**停止累加**；反算抗饱和——把饱和差值回灌到积分项。

Q9：什么是串级 PID?

- 答：两个环嵌套，**外环（如位置/速度）输出作为内环（如电流/力矩）的目标**；内环带宽更高、响应更快，外环慢。倒立摆/云台常用。

Q10：编码器测速 M 法 / T 法 / M-T 法怎么选?

- 答：**M 法**数固定时间内脉冲数，适合**高速**；**T 法**测脉冲周期，适合**低速**；**M-T 法**结合两者，全速段精度更好。

Q11：为什么说“反馈不干净，调参没有意义”?

- 答：PID 只能控制它**看到**的值。反馈被噪声污染时，误差抖动 → 输出占空比剧烈波动 → 越调越乱。所以**先保证测量质量（硬件滤波/上拉/屏蔽/降频），再谈调参。**

C. 项目必答

Q12：你的 PID 是哪种?把公式写出来。

- 答：**增量式**：`Δu = Kp(e−e1) + Ki·e + Kd(e−2e1+e2)`，`u += Δu` 后限幅。

Q13：积分分离在哪做的?阈值多少?为什么?

- 答：在 PID 计算里，当 `|e| > PID_INT_SEP_ERR` 时 `Ki` 项置 0。你项目阈值 `30`（RPM 量级）。目的：**启动/大阶跃时不积分**，防积分饱和引起超调。

Q14：输出限幅范围?为什么钳到 800?

- 答：钳在 `[0, MOTOR_DUTY_LIMIT=800]`。上限 800（80%）一是**限制最高转速**，二是避免 `CCR>ARR`（占空比 >100%）导致 L298N 双高刹车。

Q15：目标速度怎么映射成 RPM？为什么 /2？

- 答：协议速度是 0~1000 的抽象值，实测 500 ≈ 257RPM，所以近似 `target_rpm ≈ g_speed / 2`。这是**近似映射**，正式应按标定换算。

Q16：你的控制周期多少？为什么？

- 答：**20ms。**对直流减速电机足够快、又给编码器测速和计算留出稳定裕量；周期固定，避免用漂移的 dt 影响微分/积分。

Q17：你现在调参到什么程度了？

- 答：**诚实**： “代码实现并编译通过（Kp=5、Ki=1、Kd=0 为初值），但闭环实测被**编码器 PWM 噪声**阻塞，还没完成调参。” → 诚实 + 说明根因。

Q18：编码器噪声怎么处理的？根因是什么？

- 答：现象是**停转读数干净、转动时乱跳**；根因是 **PWM 开关噪声耦合进 A/B 线。**已做：内部上拉、EMA 软件平滑、PWM 降到 2kHz；**根子在硬件**，还需 RC 滤波/屏蔽/共地。

模块四：CAN

A. 基础必会

Q1：CAN 和 UART / RS485 / I2C 的区别？

- 答：CAN 是**双线差分、多主、非破坏性仲裁、错误检测强**的现场总线，适合多节点工业场景；UART 点对点；RS485 半双工主从；I2C 板内短距多从机（带地址）。

Q2：显性/隐性是什么？仲裁怎么工作？ID 大小与优先级？

- 答：总线空闲为隐性，显性电平覆盖隐性。多节点同时发送时，发隐性的节点“输了”退出，**显性的赢**，且**不破坏数据**（非破坏性仲裁）。**ID 越小优先级越高。**

Q3：CAN 有哪些帧类型？标准帧/扩展帧？

- 答：数据帧、遥控帧、错误帧、过载帧。标准帧 **11 位 ID**，扩展帧 **29 位 ID**（你项目用标准帧）。

Q4：波特率怎么来的？采样点多少？

- 答：由 1 位时间的 TQ 分段决定：`SYNC_SEG + PROP_SEG + PHASE_SEG1 + PHASE_SEG2`。采样点一般在 **70%~80%。**
- 你的配置：F103 = 36M/4/18 = 500k（采样点 88.9%），F407 = 42M/6/14 = 500k（85.7%），两端一致才能通信。

Q5：错误三状态是什么？TEC/REC/LEC 是什么？

- 答：**错误主动**（TEC<128）→ **错误被动**（128≤TEC≤255）→ **总线关闭 BusOff**（TEC>255，脱离总线）。TEC/REC 是发送/接收错误计数，LEC 是最近一次错误码。你故障注

入里见过 **LEC=3（ACK 错误）**。

Q6：为什么两端要 120Ω 终端电阻?

- 答：抑制**信号反射**并把总线拉回**隐性电平**。你项目用了分析仪自带 + 面包板远端各一个。

Q7：过滤器掩码模式 vs 列表模式？为什么 ID 要左移 5 位?

- 答：掩码按 `(收到ID & Mask) == (Filter & Mask)` 匹配一组 ID；列表逐条精确匹配。11 位 ID 在 32 位过滤器寄存器里右对齐到高位、低 5 位不参与比较，所以 `ID << 5`。你项目 `Mask=0x7FF` = 11 位全比较 = 精确接收。

Q8：bxCAN 的邮箱和 FIFO?

- 答：有 3 个**发送邮箱**，`HAL_CAN_AddTxMessage` 放进空邮箱硬件自动发送；接收有 2 个 **FIFO**，可配过滤器把不同 ID 分到不同 FIFO。

B. 进阶

Q9：非破坏性仲裁为什么不会丢数据?

- 答：因为仲裁发生在**仲裁段（ID 部分）**，输的节点立刻退出发送、转去接收，此时**还没发数据段**，所以赢家的数据不受影响。

Q10：位填充是什么？ACK 槽是什么?

- 答：为保证同步，连续 5 个相同位后插入 1 个相反位（位填充）。接收方在 **ACK 槽**回显性位表示正确收到；没人应答就是 **ACK 错误（LEC=3）**。

Q11：BusOff 怎么恢复?

- 答：`AutoBusOff=ENABLE` 时硬件自动恢复；你也注意到 **F1 HAL 错误中断会把 State 锁成 ERROR**，必须在 `ErrorCallback` 里**恢复 READY**，否则发送永远失败。

Q12：周期上报 vs 请求/应答，怎么选?

- 答：持续监测、实时性要求低 → **周期上报**（省主控）；按需控制、要确定性 → **请求/应答**（带宽按需、指令即时）。工业系统常**两者并用**（你项目就是：温度周期上报、速度命令触发）。

C. 项目必答

Q13：你的帧 ID 分表？各帧用途、DLC、周期?

- 答：0x100 心跳（8B，1s）；0x101 请求温度（1B）；0x102 温度（2B，1s）；0x103 MPU6050 加速度（6B，1s，主控暂未启用）；0x201 设置速度（3B）；0x202 速度/RPM（2B，1s）。多字节一律**小端**。

Q14：你的过滤器怎么配的？为什么用掩码?

- 答：掩码模式、`Filter` 为目标 ID、`Mask=0x7FF`（11 位全比较），实现**精确只收**该节点关心的命令帧；ID 左移 5 位对齐寄存器。主控用 Bank0 收 0x102、Bank1 收 0x202。

Q15：接收中断里做了哪些事？为什么这样设计?

- 答：中断里只 `HAL_CAN_GetRxMessage` 取帧 + `xQueueSendFromISR` 投递 + `portYIELD_FROM_ISR`；**不做解析、不打印、不加锁**。解析和状态更新放 `CanRxTask`。这就是“中断最小化”。

Q16：心跳/离线检测怎么做的？故障注入验证了什么？

- 答：记录每节点最近接收 tick，**5s 超时**判离线、恢复自动清除。故障注入（拔从节点 / 断电 / 拔主控）验证了能区分**单节点离线**和**主控 ACK 错误（LEC=3，全员离线）**。

Q17：30 分钟压力测试怎么做的？看哪些指标？结论？

- 答：三节点连续运行约半小时，观测：① 错误计数 **LEC/TEC/REC 全 0**；② **无虚假离线告警**；③ 各帧**周期计数符合设计**（心跳 1715 帧、命令 183/180 帧）。
- 结论：长时间运行**无误报、无成批丢帧。且这次测试真的抓出两个缺陷**（0x202 解析与文档不一致、离线标志无清除），定位修复后回归。

Q18：F103 和 F407 的 CAN 配置为什么不同？

- 答：**F1 没有复用功能选择寄存器**，CAN_RX（PA11）必须配 `GPIO_MODE_INPUT`（+上拉）；F4 有 AF 选择寄存器，RX/TX 都可配 `AF_PP + AF9`。照抄 F4 会导致 F1 CAN 初始化卡死（困扰两天的坑）。

模块五：Bootloader / Flash

A. 基础必会

Q1：IAP 和 ISP 的区别？

- 答：**ISP** 用芯片自带 Bootloader（串口/USB）烧录，要动 boot 引脚/复位；**IAP** 是应用自己实现 Bootloader，**运行中升级 APP**，无需额外硬件操作（你做的）。

Q2：为什么需要 Bootloader？

- 答：支持**现场/远程升级（OTA）**、固件校验、双区/回滚，产品出厂后不用拆机接调试器。

Q3：Flash 的物理特性？

- 答：写操作只能把位 **1→0**，要把 0 变回 1 必须**擦除**；所以**先擦后写。**

Q4：STM32F407（1MB）的扇区布局？

- 答：Sector 0–3 = 各 **16KB**；Sector 4 = **64KB**；Sector 5–11 = 各 **128KB。**合计 64+64+896 = **1024KB。**

Q5：擦除的最小单位是什么？一次最多擦多少？

- 答：**最小单位是一个扇区**，不能按字节、也不能小于一个扇区擦。**单次扇区擦除最多 128KB**（大扇区），最小 16KB；整片全擦也就 1MB。擦写寿命典型约 1 万次。

Q6：写的最小单位？

- 答：F4 内部 Flash 按 **64 位双字**（doubleword）编程（HAL 用 `FLASH_TYPEPROGRAM_DOUBLEWORD`）。你 `BlFlashWriteRange` 就是把字节攒成 64 位再写、尾块补 0xFF。

Q7：为什么分区要按扇区边界对齐?

- 答：因为擦除只能整扇区擦。分区边界若落在扇区中间，擦 A 区会**误擦 B 区**。所以 Bootloader 占整扇区，APP 从扇区边界开始。

B. 项目必答

Q8：你的 Flash 怎么分区的?

- 答：**Bootloader = Sector 0–3 = 64KB**，起始 `0x08000000`；**APP = Sector 4–11 = 960KB**，起始 `0x08010000`（正好是 Sector 4 起始，扇区对齐）。

Q9：怎么从 Bootloader 跳到 APP?

- 答：① 关中断；② 读 APP 首地址的初始 **MSP** 并 `__set_MSP()`；③ 取 `APP_START+4` 的 **Reset_Handler** 地址，用**函数指针**调用跳转（永不返回）。

Q10：为什么 APP 要设置 VTOR?

- 答：APP 的向量表搬到了 `0x08010000`，若不设 `SCB->VTOR`，中断仍按 `0x08000000` 查表，会跳到错误地址导致 HardFault。

Q11：怎么判断 APP 有效? magic 放在哪?

- 答：判断两件事：APP 首字的**栈指针在 SRAM 范围**内，且 `APP_START + 0x100` 处的 **magic = 0xA5A5A5A5**。升级完成且校验通过才写 magic；不通过就不写，下次启动报 `NO APP`，**不会跑损坏固件**。

Q12：升级协议的帧格式和命令?

- 答：帧 = `A5 5A | CMD(1B) | ADDR(4B 小端) | LEN(2B 小端) | DATA(≤256B) | CRC32(4B 小端，仅 DATA)`。
- 命令：**ERASE**（按总长擦）、**WRITE**（写一帧）、**FINISH**（总长 + 全镜像 CRC 校验）、**REBOOT**（复位）。每帧回 **ACK(0x06)** 或 **NACK(0x15)**，超时 500ms、重试 3 次。

Q13：单帧 CRC 和全镜像 CRC 分别防什么?

- 答：**单帧 CRC** 防传输噪声改坏某一帧；**全镜像 CRC**（FINISH 时对整个 Flash 重算）防**漏帧/乱序/帧都合法但整体不对**。

Q14：CRC32 原理? 和简单校验和有什么区别? 怎么保证两端一致?

- 答：CRC32 按多项式 `0xEDB88320`（IEEE 802.3，反射）逐位求余，对**任意位翻转**检出能力强；校验和只是相加，对特定错误不敏感。
- 两端一致靠**同一算法**：固件用逐位反射实现，上位机用 Python `zlib.crc32`，并用标准向量 `crc32("123456789")==0xCBF43926` 验证一致。

Q15：升级中途断电会不会变砖?

- 答：**不会。**先擦后写，断电会让 APP 区不完整；但因为 **magic 未写**，Bootloader 判定 APP 无效 → 报 `NO APP`，可**重新升级恢复。**

Q16：上位机（Python）整个流程?

- 答：等 `BOOT` 横幅 → 发 `u` 进升级模式 → **ERASE**（带镜像总长）→ 按 **256 字节分包 WRITE**（地址递增）→ **FINISH**（总长 + 全图 CRC32）→ **REBOOT。**用 `pyserial`，每帧等 ACK、超时重传。

Q17：为什么镜像总长放在 DATA 里，而不是 LEN 字段?

- 答：LEN 只有 **2 字节，最大 65535**，装不下 960KB 的镜像。所以 ERASE/FINISH 用 DATA 携带 **4 字节总长**；WRITE 的 LEN 只表示**单帧块长**（≤256），2 字节够用。

Q18：WRITE 为什么要校验地址范围?

- 答：防止脏帧/错帧把数据写到 **Bootloader 区**或其他非法地址，导致系统变砖。只允许写 `[APP_START, APP_START+APP_LEN_MAX)`。

附：一页速记（面试前扫一眼）

类别	关键数字 / 结论
MCU	F407 主控 + 2×F103 从节点
总线	CAN 500kbps，11 位标准帧
帧 ID	0x100 心跳 / 0x101 请求 / 0x102 温度 / 0x103 加速度 / 0x201 设速 / 0x202 RPM
离线检测	心跳 1s，超时 5s
压力测试	约 30 分钟，LEC/TEC/REC=0，心跳 1715 帧，命令 183/180 帧，无误报
RTOS	FreeRTOS，队列传帧、互斥量保护打印、heap_4、优先级继承、FromISR
PID	增量式，Kp=5/Ki=1/Kd=0，积分分离阈值 30，20ms，限幅 [0,800]
编码器	TIM3 4 倍频，PPR=1079，EMA 平滑
Flash	扇区 16/64/128KB；擦除最小单位=扇区，最大 128KB
Bootloader	64KB@0x08000000；APP 960KB@0x08010000；magic 0xA5A5A5A5@+0x100
升级协议	A5 5A|CMD|ADDR|LEN|DATA|CRC32；ERASE/WRITE/FINISH/REBOOT；ACK 0x06/NACK 0x15
CRC32	IEEE 802.3，0xEDB88320，校验向量 0xCBF43926

怎么用这份题库（建议）

1. **先自测**：每个模块从“项目必答”开始，遮住答案口述，卡壳的标记出来。
2. **再回看代码**：标记的题回到 `firmware/` 对应 `.c` 文件，对着代码把答案讲一遍。
3. **最后压测**：让同学/AI 当面试官随机抽题，**要求你不看稿口头回答。**
4. 目标不是背答案，而是**能用自己的话讲出“为什么这么设计”。**